

Design Procedure

There were two tasks in this lab. We were tasked with implementing algorithms into hardware in Verilog. The first being an ASMD chart for a bit counting. The second being a block diagram for a binary search algorithm. For all of these tasks we created datapaths and controllers.

Task #1

Starting out we first understood how the Bit Counting Algorithm functioned. We started with making the controller and datapath. The controller follows the logic of the ASMD chart and sets variables according to relevant actions. The datapath is fed with those control signals and then makes the changes. Finally, we implemented them in our top module. The logic for our controller involved when: A is empty, right shift A, load A, set done, initialize result, and increment the result. We changed these control signals according to the ASMD chart.

Task #2

We first drew our ASMD chart for task 2 (Figure 1). We had four states. S0 which is our beginning state (idle). We check if start (s) and if so, we initialize our data. In this case it is the addr, location (loc), counter for incr/decr (count), and whether a match to A has been found (found). We then have S_wait. The reason we kept this state is since we have registers it will take one clock cycle for the mif data to be accessed from the RAM. This state acts as the one clock cycle buffer. Next our biggest state is S_start. The first thing we check is if the RAM addr (our data) is equal to A. If so, we set found = 1 and the loc to our addr. If RAM addr wasn't equal to A then we check how much our count is. If count > 4 then we know the range has been exhausted and that RAM doesn't contain the value so we have it go to Done state. If however our count is valid then we check whether A was bigger than addr. If it was then we increment our addr (and count increase), if not we decrement our addr (and increase count). Then it goes back into the wait state as we need to wait for the next RAM data from mif. Our final state is S_Done. This just has the output Done and then we go back to our first state.

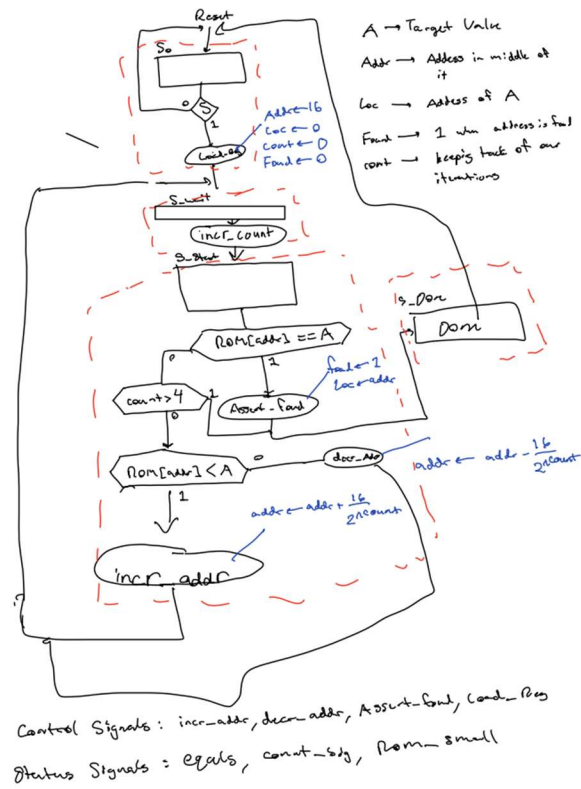


Figure 1: ASMD Chart for Task 2

Overall System: Top Level Block Diagram

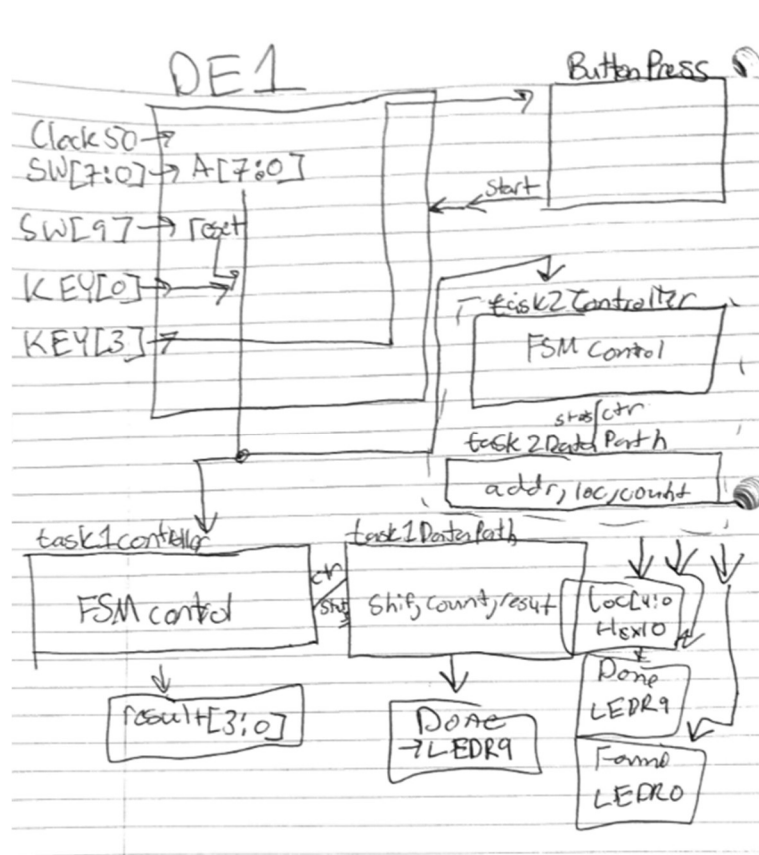


Figure 2: Top Level Block Diagram

Results

Our system implements two tasks on the DE1-SoC, switchable using SW9. In Task 1, the circuit counts the number of 1-bits in the 8-bit input A on SW7–SW0, displays the result on HEX0, and lights LEDR9 when finished. In Task 2, the circuit performs a binary search over a sorted 32-element array stored in on-chip RAM. The user specifies a target value on SW7–SW0 and starts the search with KEY3. When complete, LEDR9 signals Done, LEDR0 indicates whether the value was Found, and HEX1–HEX0 display the memory address where A was located. Both tasks freeze in their completed state until a reset via KEY0 is issued, returning all registers to their initial values.

Task1Controller



Figure 3: The ModelSim waveforms to Task1Controller module.

In the upper image, we start by testing reset. Then we set $A0 = 1$ to see if the `incr_result` and `right_shiftA` is asserted. We can see that as soon as $A0 = 1$ they both go to one. We then set $A0$ to 0 and `incr_result` goes to 0 but not `right_shiftA`. This is the intended result as `set_done` is still 0. We let this run for two more clock cycles alternating the value of A (this simulates an actual test). Finally, we set $A_empty = 1$. This makes `set_done` = 1. And stops our `incr_result` and `right_shiftA` = 0. With these tests we can confidently say our controller works as it is intended.

Task1Datapath

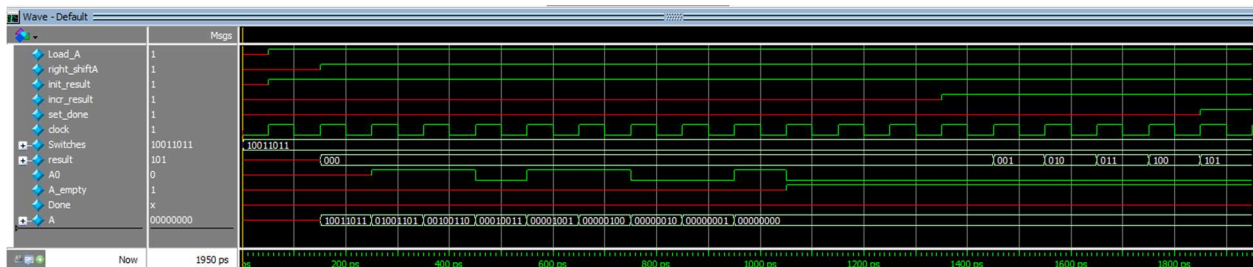


Figure 4: The ModelSim waveforms to Task1Controller module.

In the upper in Figure 4, we started by setting all our switches to 10011011. Note it takes a few clock cycles for them to actual be loaded into A . Its asserts on the second clock cycle because we need to still load A . We then set $Load_A$ to 1 and `init_result` to 1. The next clock cycle we set `right_shiftA` = 1 and ran 12 times. This simulates the actual process our module would do. We made it 12 times even though it would only need 7 clock cycles to test behaviour with all 0s. A_empty updates where intended. We can see $A0$ updating as specified and double check it with our changing A . Note our result doesn't change (as intended). Now we set `incr_result` to 1 and ran 5 times. This is it takes for our result to increase to the right value and then we set `set_done` to 1, confirming our datapath works as intended.

buttonPress

This module is to handle the metastability that occurs with having two buttons. All it does is when the button pressed it aligns it as a next state condition making sure it stays on clock cycle. To test this we check for a button being pressed one clock cycle and held down(Figure 5).

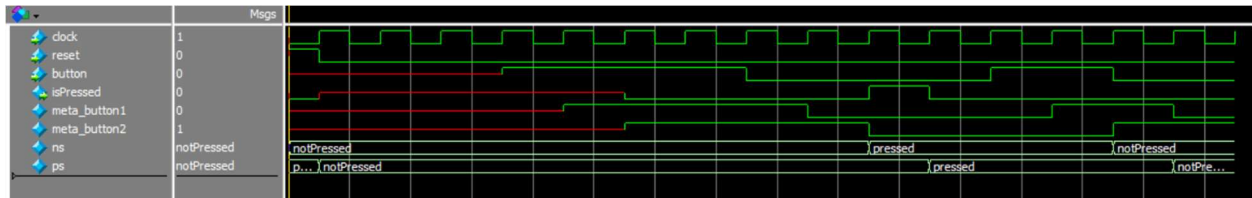


Figure 5: buttonPress ModelSim

task2Controller

Looking at the bottom we started with reset. Then looped back to s0 by having start go to 0 then to 1 to begin. Load Reg is activated meaning that our variables needed to read through the file have been created. It turns off next clock cycle. The next clock cycles our count increases as we move through the wait state into the S-start state (also need to load from MIF). We then see the ps and ns cycle between start and wait. Rom small is active for one cycle meaning the RAM was smaller than A. It turns off but incr_count keeps going meaning that for the rest of the time the RAM is bigger than A. Finally, once the value is found the assert done is true and our ps becomes done and our ns becomes started.

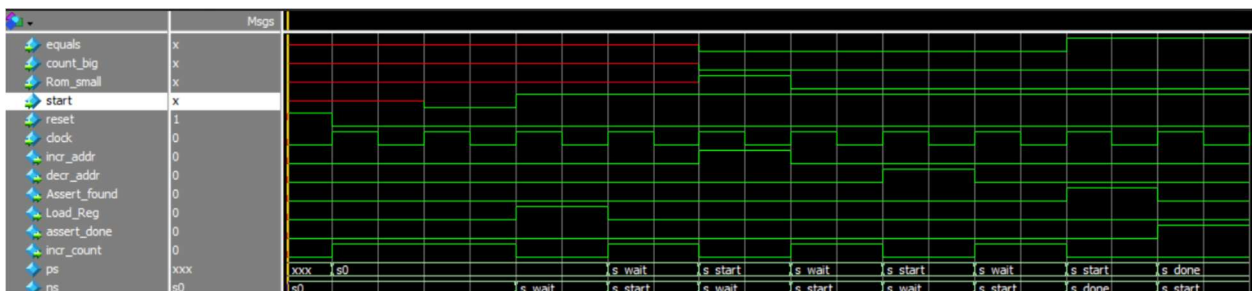


Figure 6: ModelSim for task2Controller

task2DataPath

Looking below for our simulation of task2DataPath. We start by loading the register (Load_Reg). It takes a little as our datain comes in along with count. (NOTE equals is set

high as in the system once equal is reached the module is done so there is no point of having equal go to 0 unless in reset [implementation design]). We see that our datain changes aptly all the way to 12 [equals would change as it does on board]. Then we manually decr and incr to test function and change A to 11. Once it reaches A it stops as intended. And doesn't continue as count > 4. Finally, when assert found is true loc changes to the value of datain.

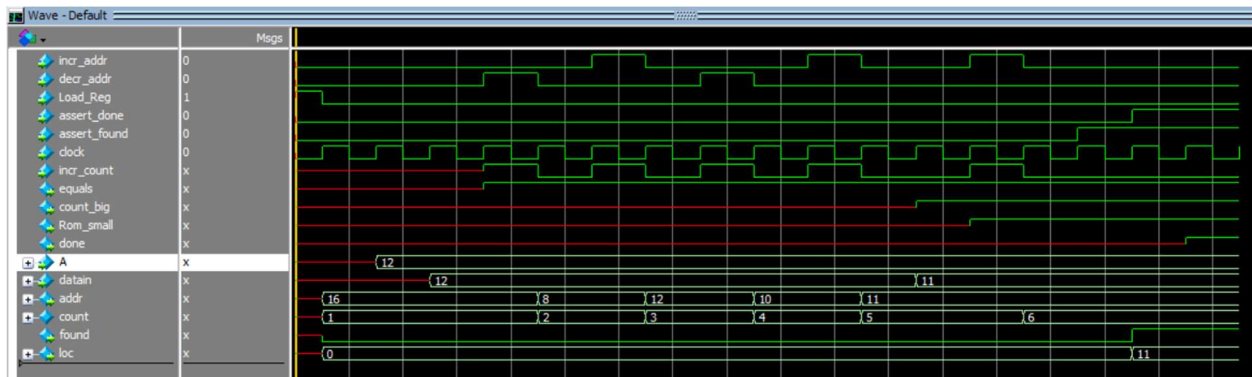


Figure 7: The ModelSim for task2DataPath

DE1_SoC

The top-level waveform is shown in Figures A and B. The top-level module instantiates both the Task 1 bit-counting circuit and the Task 2 binary search circuit, switching between them using SW9 (mode). When mode is low, start_task1 is asserted and the bit-counter runs; when mode is high, start_task2 is asserted and the binary search runs. Because a buttonPress module is used to handle metastability the press is delayed by two clock cycles. The reset signal is active-high and from ~KEY[0]. In the first waveform (Figure 8), we verify Task 1 and Task 2 interconnection. After reset, the datapath registers (addr, count, loc, found, done) initialize, and A is read from SW7–SW0. We can observe start_task1 and start_task2 toggling correctly based on mode, confirming the switching logic is wired properly. The dataOut signal from the RAM responds to changes in addr after one clock cycle for latency. The hex_value output reflects the correct address location once the binary search completes and done is asserted, with LEDR9 going high and LEDR0 reflecting the found. In the second waveform (Figure 9), we look at the full binary search execution. Starting from reset, addr initializes to 16 (the midpoint of the 32-element array). After start is pulsed, the controller begins iterating: addr steps to 8, then 12, then 10, as seen in the addr trace converging toward the target value. The ROM_small and equals status signals update one cycle after each address change, consistent with the one-cycle RAM read latency that necessitates a wait state in the ASMD chart. Once the target is located,

The screenshot shows a logic analyzer interface with a list of signals on the left and a corresponding waveform on the right. The signals are:

- clock_50**: 0
- HE0**: 1000000
- HE1**: 1000000
- HE2**: 1111111
- HE3**: 1111111
- HE4**: 1111111
- HE5**: 1111111
- HE6**: 1000000
- HE7**: 1111111
- SW**: 0111111111
- reset**: 0
- start**: 0
- set_done**: 0
- res_result**: 0
- load_A**: 1
- res_result**: 0
- right_shiftA**: 0
- Done**: 0
- count**: 0
- AD**: 1
- A**: 255
- A_empty**: 0
- mode**: 1
- start_task1**: 0
- start_task2**: 0
- count**: 0
- ptr_addr**: 0
- decr_addr**: 0
- assert_bound**: 0
- load_flag**: 0
- count_flag**: 1
- ROM_small**: 1
- done**: 1
- assert_done**: 0
- ptr_count**: 1
- band**: 0
- task**: 23
- loc**: 00000
- dataOut**: 23
- hex_value**: 00000000

The waveform shows the digital signals over time. The 'Done' signal is high for most of the duration. The 'count' signal is high for most of the duration. The 'ptr_count' signal is high for most of the duration. The 'dataOut' signal is high for most of the duration. The 'hex_value' signal is high for most of the duration. The time scale is in microseconds (µs).

Figure 9: Binary search from top level

Flow Summary

Flow Summary	
<<Filter>>	
Flow Status	Successful - Sun May 10 17:51:30 2026
Quartus Prime Version	17.0.0 Build 595 04/25/2017 SJ Lite Edition
Revision Name	DE1_SoC
Top-level Entity Name	DE1_SoC
Family	Cyclone V
Device	5CSEMA5F31C6
Timing Models	Final
Logic utilization (in ALMs)	N/A
Total registers	36
Total pins	67
Total virtual pins	0
Total block memory bits	256
Total DSP Blocks	0
Total HSSI RX FCSs	0
Total HSSI FMA RX Deserializers	0
Total HSSI TX FCSs	0
Total HSSI FMA TX Serializers	0
Total PLLs	0
Total DLLs	0

Experience Report

We found this lab to be straightforward, but we struggled a lot with the logic. The main problem was also with our top level and making sure SW9 alternated between tasks. The unsigned binary also was confusing. Finally, incr and decr the RAM [addr] was also confusing.

This lab took us approximately 12 hours, broken down as follows:

- Reading – 15min
- Planning – 15min
- Design – 30min
- Coding – 5hr
- Testing – 2hr
- Debugging – 4hr